

Lab 4 Report: Prefix Sum and Hashing

Name: Cao Bảo Khương

ID: ITDSIU22176

Course: Data Structures and Algorithms

Part A: Guided Practice

Task A1: Pattern Identification

1. **Given an array and many queries (L,R), return the sum of each range.**
 - **Technique:** Prefix Sum.
 - **Justification:** Since there are multiple queries on the same array, precomputing prefix sums allows each range query to be answered in $O(1)$ time using $P[R] - P[L - 1]$.
2. **Determine whether an array contains a zero-sum subarray.**
 - **Technique:** Prefix Sum + Hashing (HashSet).
 - **Justification:** A zero-sum subarray exists if the same prefix sum value appears twice (or if a prefix sum is zero). A HashSet provides $O(1)$ lookup for previously seen prefix sums.
3. **Count how many subarrays have sum equal to k.**
 - **Technique:** Prefix Sum + Hashing (HashMap<PrefixSum, Frequency>).
 - **Justification:** We need to find how many times ($CurrentPrefixSum - k$) has occurred previously. Storing the frequency of each prefix sum allows us to count all such occurrences in one pass.
4. **Find the longest subarray with sum equal to k.**
 - **Technique:** Prefix Sum + Hashing (HashMap<PrefixSum, FirstIndex>).
 - **Justification:** To maximize length, we need the *earliest* index where a specific prefix sum appeared. Storing the first occurrence index allows us to calculate $currentIdx - firstIdx$.
5. **Find the minimum length subarray whose sum is at least S when all values are positive.**
 - **Technique:** Sliding Window.
 - **Justification:** Since all values are positive, the window sum is monotonic (adding elements increases sum, removing elements decreases it). Sliding window is more space-efficient ($O(1)$) than prefix sum for this specific condition.
6. **Find the longest substring without repeating characters.**
 - **Technique:** Sliding Window + Hashing (HashSet or Map for characters).
 - **Justification:** This is a classic “best valid window” problem. We expand the window until a duplicate is found, then shrink from the

left.

Task A2: What Should Be Stored?

Problem 2: Zero-Sum Subarray Exists - Prefix Sum Meaning: The cumulative total from index 0 to i . - **What is stored:** A `HashSet<Long>` containing all unique prefix sums encountered so far. - **Why it is enough:** If we encounter a prefix sum that is already in the set, it means the elements between the first occurrence and the current index sum to zero. - **Condition checked:** `if (set.contains(currentSum) || currentSum == 0)`

Problem 3: Count Subarrays with Sum k - Prefix Sum Meaning: The cumulative total from index 0 to i . - **What is stored:** A `HashMap<Long, Integer>` where keys are prefix sums and values are their frequencies. - **Why it is enough:** For any index i , if a subarray ending at i has sum k , then $CurrentSum - EarlierSum = k$, or $EarlierSum = CurrentSum - k$. The frequency tells us how many such $EarlierSum$ positions exist. - **Condition checked:** `count += map.getOrDefault(currentSum - k, 0)`

Problem 4: Longest Subarray with Sum k - Prefix Sum Meaning: The cumulative total from index 0 to i . - **What is stored:** A `HashMap<Long, Integer>` where keys are prefix sums and values are the *first* index they appeared at. - **Why it is enough:** To maximize $i - j$, we need the smallest possible j such that $PrefixSum[i] - PrefixSum[j] = k$. Storing only the earliest index ensures we find the maximum length. - **Condition checked:** `if (map.containsKey(currentSum - k)) maxLen = Math.max(maxLen, i - map.get(currentSum - k))`

Task A3: Brute Force vs. Optimized

Problem 4: Longest Subarray with Sum k - Brute-Force Idea: Iterate through all possible subarrays (i, j) using nested loops ($O(n^2)$), calculate the sum of each ($O(n)$ or $O(1)$ with optimization), and keep track of the maximum length among those that sum to k . - **Optimized Idea:** Use a prefix sum and a `HashMap` to store the first occurrence of each prefix sum. In a single pass ($O(n)$), for each current sum S , check if $S - k$ was seen before. - **Why it avoids repeated work:** Instead of re-calculating sums for every subarray starting at different points, it uses the mathematical relationship between cumulative sums to “look back” at all valid starting points simultaneously. - **Complexity:** - **Brute Force:** $O(n^2)$ time, $O(1)$ space. - **Optimized:** $O(n)$ time, $O(n)$ space.

Problem B1: Range Sum Query - Brute-Force Idea: For every query (L, R) , run a loop from L to R and sum the elements. - **Optimized Idea:** Precompute a prefix sum array P where $P[i] = arr[0] + \dots + arr[i]$. Then answer each query using $P[R] - P[L - 1]$ (with $P[-1] = 0$). - **Why it avoids repeated**

work: It calculates each element's contribution to the total once, rather than summing the same middle elements for overlapping queries. - **Complexity:** - **Brute Force:** $O(Q \times N)$ time, $O(1)$ space (where Q is number of queries). - **Optimized:** $O(N + Q)$ time, $O(N)$ space.

Part B: Classical Exercises

Exercise B1: Range Sum Query

- **Problem Restatement:** Given an array and queries (L, R) , return the sum of elements from L to R .
- **Pattern:** Prefix Sum.
- **Input/Output:**
 - Input: `int[] nums, int L, int R`
 - Output: `long` (the sum)
- **Assumptions & Edge Cases:**
 - $L = 0$ is valid.
 - $L = R$ is a valid subarray of length 1.
- **Brute-Force Idea:** Loop from L to R for every query. $O(N)$ per query.
- **Optimized Idea:** Precompute prefix sums $P[i]$. Result is $P[R+1] - P[L]$. $O(1)$ per query.
- **What is stored:** A `long[]` array where index i stores sum of first i elements.
- **Invariant:** $P[i] = \sum_{j=0}^{i-1} \text{nums}[j]$.
- **Complexity:** $O(N)$ preprocessing, $O(1)$ query. $O(N)$ space.
- **Test Cases:**
 1. `[1, 2, 3]`, range `[0, 2]` -> Expected 6.
 2. `[1, 2, 3]`, range `[1, 1]` -> Expected 2.
 3. `[1, 2, 3]`, range `[0, 1]` -> Expected 3.

Exercise B2: Zero-Sum Subarray Exists

- **Problem Restatement:** Check if any contiguous subarray sums to 0.
- **Pattern:** Prefix Sum + HashSet.
- **Input/Output:**
 - Input: `int[] nums`
 - Output: `boolean`
- **Brute-Force Idea:** Check every (i, j) pair, calculate sum. $O(N^2)$.
- **Optimized Idea:** If $\text{PrefixSum}[i] == \text{PrefixSum}[j]$, the sum between them is 0.
- **What is stored:** `HashSet<Long>` of unique prefix sums.
- **Invariant:** If a prefix sum repeats, the subarray between occurrences has net change 0.
- **Complexity:** $O(N)$ time, $O(N)$ space.
- **Test Cases:**

1. {4, 2, -3, 1, 6} -> Expected `true`.
2. {1, 2, 3} -> Expected `false`.
3. {0, 1} -> Expected `true`.

Exercise B3: Subarray Sum Equals k (Existence)

- **Problem Restatement:** Check if any subarray sums to k .
- **Pattern:** Prefix Sum + HashSet.
- **Brute-Force Idea:** Nested loops for all subarrays. $O(N^2)$.
- **Optimized Idea:** Check if `currentSum - k` exists in previous sums.
- **What is stored:** HashSet<Long> of seen prefix sums.
- **Complexity:** $O(N)$ time, $O(N)$ space.
- **Test Cases:**
 1. {1, 2, 3}, $k=3$ -> `true`.
 2. {1, 2, 3}, $k=7$ -> `false`.
 3. {-1, 1, 0}, $k=0$ -> `true`.

Exercise B4: Count Subarrays with Sum k

- **Problem Restatement:** Count how many subarrays sum to k .
- **Pattern:** Prefix Sum + HashMap<PrefixSum, Frequency>.
- **What is stored:** HashMap<Long, Integer> (PrefixSum -> Count).
- **Invariant:** `map.get(currentSum - k)` tells us how many previous points satisfy the condition.
- **Complexity:** $O(N)$ time, $O(N)$ space.
- **Test Cases:**
 1. {1, 1, 1}, $k=2$ -> 2.
 2. {1, 2, 3}, $k=3$ -> 2.
 3. {1, -1, 0}, $k=0$ -> 3.

Exercise B5: Longest Subarray with Sum k

- **Problem Restatement:** Find max length of a subarray summing to k .
- **Pattern:** Prefix Sum + HashMap<PrefixSum, FirstIndex>.
- **What is stored:** HashMap<Long, Integer> (PrefixSum -> Earliest Index).
- **Invariant:** We only update the map for a prefix sum if it hasn't been seen, ensuring the "earliest" position is kept.
- **Complexity:** $O(N)$ time, $O(N)$ space.
- **Test Cases:**
 1. {1, -1, 5, -2, 3}, $k=3$ -> 4.
 2. {1, 2, 3}, $k=6$ -> 3.
 3. {-2, -1, 2, 1}, $k=1$ -> 2.

Exercise B6: Pivot Index / Equilibrium Index

- **Problem Restatement:** Find index where $LeftSum == RightSum$.

- **Pattern:** Prefix Sum reasoning.
 - **Explanation:** Calculate total sum first. Then $RightSum = TotalSum - LeftSum - CurrentElement$.
 - **Complexity:** $O(N)$ time, $O(1)$ space.
 - **Test Cases:**
 1. {1, 7, 3, 6, 5, 6} -> 3.
 2. {1, 2, 3} -> -1.
 3. {2, 1, -1} -> 0.
-

Part D: Reflection and Explanation

Task D1: Recognition Reflection

1. **When does Sliding Window work well?**
It works well for finding a “best valid window” (shortest/longest) when the condition is monotonic (e.g., all positive numbers, non-decreasing sums).
2. **Why can negative numbers make Sliding Window difficult?**
With negative numbers, adding an element can decrease the sum and removing one can increase it. This breaks the simple logic of “expand when sum is too small, shrink when too large.”
3. **What does a prefix sum represent in your own words?**
It’s a “running total” that summarizes the history of the array up to that point. It’s like a snapshot of the cumulative state.
4. **What is the difference between storing existence, frequency, and earliest position?**
 - **Existence:** Answers “Can we reach this sum?”.
 - **Frequency:** Answers “How many ways can we reach this sum?”.
 - **Earliest Position:** Answers “What is the largest distance between two points that produce this sum difference?”.
5. **Which problem was hardest to classify?**
“Count subarrays divisible by k ” was slightly harder because it requires thinking about remainders and handling negative modulo results in Java.

Task D2: Explain Without Code (Exercise B5)

To find the longest subarray that sums to k , we use a “running total” as we walk through the array. Imagine you have a map where you write down the first time you ever saw a particular total. At each step, if your current total is S , you check your map for $S - k$. If it’s there, the space between that first occurrence and your current position is a subarray that sums exactly to k . We keep track of the biggest gap we find. By only recording the *first* time we see a total, we guarantee that the gap is as wide as possible. This approach is efficient because we only look at each number once and look up our history in an instant.